

Proper Notes Format

Perfect. For your notes, I'll use:

- ♦ Topic Name
- Lots of spacing
- Deep explanations
- No compression
- No `java`, `json`, `http` labels before code
- Same teaching style as your notes

♦ PACKAGE DECLARATION

```
package com.scs.emailthreat.controller;
```

What is a package?

A package is used to organize Java classes into folders.

Think of it like folders on your computer.

Example:

```
Documents
├── Projects
├── Photos
└── Videos
```

Folders help keep files organized.

Java packages do the same thing for classes.

package Keyword

```
package com.scs.emailthreat.controller;
```

Here:

```
package
```

tells Java:

```
"This class belongs to a package."
```

com.scs.emailthreat.controller

This is the package name.

Break it down:

```
com
```

Usually represents a company or organization.

```
scs
```

Project or organization name.

```
emailthreat
```

Project name.

```
controller
```

Folder containing controller classes.

So Java expects this file to be inside:

```
src/main/java/com/scs/emailthreat/controller
```

Why use packages?

Without packages:

```
EmailController
EmailService
EmailRequestDTO
EmailResponseDTO
UserController
AdminController
```

would all sit in one location.

As projects grow, this becomes messy.

Packages help organize code.

Example:

```
controller
├── EmailController

service
├── EmailService

dto
├── EmailRequestDTO
├── EmailResponseDTO
```

Much cleaner and easier to maintain.

◆ IMPORT STATEMENTS

```
import com.scs.emailthreat.dto.EmailRequestDTO;
import com.scs.emailthreat.dto.EmailResponseDTO;
import com.scs.emailthreat.service.EmailService;
```

Why imports are needed

Suppose EmailController is inside:

```
controller
```

and EmailRequestDTO is inside:

```
dto
```

They are in different packages.

Java cannot automatically see classes from other packages.

So we use:

```
import
```

to tell Java:

"I want to use this class from another package."

EmailRequestDTO

```
import com.scs.emailthreat.dto.EmailRequestDTO;
```

Used to receive incoming data.

Example request:

```
{  
  "emailText": "You won a lottery"  
}
```

Spring converts that JSON into:

```
EmailRequestDTO
```

object.

EmailResponseDTO

```
import com.scs.emailthreat.dto.EmailResponseDTO;
```

Used to send data back to the client.

Example response:

```
{  
  "result": "SPAM",  
  "confidence": 1.0  
}
```

This response comes from:

EmailResponseDTO

EmailService

```
import com.scs.emailthreat.service.EmailService;
```

Contains business logic.

The controller receives requests.

The service performs the actual work.

Example:

```
emailService.analyze(...)
```

The controller says:

"EmailService, analyze this email for me."

Simple Analogy

Think of a restaurant.

```
Customer
  ↓
Waiter
  ↓
Chef
```

Customer:

User

Waiter:

Controller

Chef:

Service

The waiter takes the order.

The chef prepares the food.

Similarly:

Controller receives requests.

Service performs the actual work.

◆ SPRING IMPORTS

```
import org.springframework.http.ResponseEntity;
```

This import gives access to:

```
ResponseEntity
```

class.

What is ResponseEntity?

ResponseEntity is a Spring class.

It represents a complete HTTP response.

An HTTP response contains:

```
Status Code
+
Response Body
```

Example:

```
200 OK
```

and

```
{
  "result": "SPAM",
  "confidence": 1.0
}
```

Together these form an HTTP response.

ResponseEntity lets us control both.

Example

```
return ResponseEntity.ok(
    new EmailResponseDTO("SPAM", 1.0)
);
```

Spring sends:

```
Status: 200 OK
```

and

```
{
  "result": "SPAM",
  "confidence": 1.0
}
```

back to the client.

What does ok() mean?

```
ResponseEntity.ok(...)
```

means:

"Everything worked successfully."

Spring automatically sets:

```
200 OK
```

Other Common Methods

```
ResponseEntity.ok(...)
```

Returns:

```
200 OK
```

```
ResponseEntity.badRequest()
```

Returns:

```
400 Bad Request
```

```
ResponseEntity.status(500)
```

Returns:

```
500 Internal Server Error
```

Easy Way to Remember

Think of:

```
ResponseEntity
```

as a box.

Inside the box:

```
Status Code
+
Data
```

Example:

```
Status = 200 OK
result = SPAM
```

confidence = 1.0

That box is the ResponseEntity.

◆ WEB ANNOTATION IMPORTS

```
import org.springframework.web.bind.annotation.*;
```

The star:

```
*
```

means:

"Import everything from this package."

This package contains many annotations:

```
@RestController
@RequestMapping
@PostMapping
@GetMapping
@RequestBody
@CrossOrigin
@PathVariable
@RequestParam
```

Annotations are special instructions given to Spring.

They tell Spring what to do.

For example:

```
@RestController
```

means:

"Treat this class as a REST API controller."

```
@RequestMapping("/email")
```

means:

"This controller handles URLs starting with /email."

```
@PostMapping("/analyze")
```

means:

"Run this method when a POST request comes to /analyze."

Why use * ?

Instead of:

```
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.CrossOrigin;
```

we can simply write:

```
import org.springframework.web.bind.annotation.*;
```

which imports all of them.

In Your Project

These all come from:

```
import org.springframework.web.bind.annotation.*;
```

```
@RestController
@RequestMapping("/email")
@CrossOrigin("")
@PostMapping("/analyze")
@RequestBody
```

Summary

- package → tells Java where the class belongs.
- import → allows using classes from other packages.
- EmailRequestDTO → receives request data.
- EmailResponseDTO → sends response data.
- EmailService → contains business logic.
- ResponseEntity → controls status code and response body.
- web.bind.annotation.* → provides annotations used by Spring Boot.
- Annotations are instructions that tell Spring how to handle requests.

This is a good stopping point for **Part 1**. The next section should start with:

◆ @RestController and @RequestMapping("/email") Deep Dive

because that's where Spring starts registering your controller and building the routing table.